PROGRAMACIÓN DE SERVICIOS Y PROCESOS - 2º DAM

I.E.S. ALONSO DE AVELLANEDA (ALCALÁ DE HENARES)
EVALUACIÓN: 1ª Evaluación | **FECHA:** Convocatoria Ordinaria
NOMBRE Y APELLIDOS: _____
CALIFICACIÓN: _____ / 10

Esta prueba evalúa los siguientes resultados de aprendizaje:

- **RA1:** Desarrolla aplicaciones compuestas por varios procesos reconociendo y aplicando principios de programación paralela.
- **RA2:** Desarrolla aplicaciones compuestas por varios hilos de ejecución analizando y aplicando librerías específicas del lenguaje de programación (`Semaphore`, `Timer`, `TimerTask`, `join`, `synchronized`).

Duración:

- **Tiempo estándar:** 115 minutos (1 hora y 55 minutos).
- **Adaptación NEAE (+10%):** 126 minutos (2 horas y 6 minutos).

TAREAS A REALIZAR

Una empresa de monitorización medioambiental precisa un prototipo automatizado para recoger y procesar los niveles de calidad del aire emitidos por una red de estaciones sensoras distribuidas en una ciudad.

Se consideran tres fases durante la ejecución del sistema:

1. **Despliegue y toma de muestras:** Lanzamiento de procesos independientes para cada estación sensora y generación de medidas.
2. **Procesamiento concurrente con control de concurrencia y monitorización periódica:** Lectura multihilo de las muestras recogidas, regulada mediante un semáforo de acceso limitado y supervisada en tiempo real mediante un temporizador (`Timer` / `TimerTask`).
3. **Consolidación y muestra de resultados:** Agregación estadística en sección crítica y presentación final por pantalla.

La solución se divide en **dos aplicaciones independientes** que interactúan entre sí:

---

1. Aplicación Central (Servidor de Monitorización)

- **Entrada de datos:** Recibe el número de estaciones sensoras a desplegar (procesos hijo) como argumento de línea de comandos (`args[0]`) o solicitado por teclado (`Scanner`) al iniciar. Si el valor no es un número entero válido o es menor o igual a cero, se establecerá un valor de **5 estaciones por defecto**.
- **Gestión de procesos hijo:** Crea los procesos hijo de las estaciones sensoras (bien secuencialmente o de forma concurrente con `ProcessBuilder`).
    - Debe ser compatible tanto con Linux (`/usr/bin/java` o consulta con `which java`) como con Windows (`where java` / variable en `PATH`).
    - Lee por la salida estándar del proceso hijo su identificador único (`id_estacion`) y lo almacena en una estructura en memoria.
- **Sincronización de procesos:** El programa principal no continuará con el procesamiento hasta que hayan concluido todos los procesos de las estaciones sensoras (usando `waitFor()`).
- **Temporizador de supervisión (`Timer` y `TimerTask`):**
    - Nada más comenzar la fase de procesamiento de ficheros, la aplicación principal activará un `Timer` con una tarea `TimerTask`.
    - La tarea se ejecutará cada segundo (**delay de 1000 ms y periodo de 1000 ms**), mostrando por pantalla el tiempo transcurrido, los hilos de lectura actualmente activos y los permisos libres en el semáforo.
    - Una vez finalizada la lectura de todos los datos por parte de los hilos, el hilo principal deberá detener obligatoriamente el temporizador invocando **`timer.cancel()`** para liberar los recursos y permitir que la JVM finalice limpiamente.
- **Lectura multihilo y Semáforo (`Semaphore`):**
    - Se lanzará **un hilo de lectura por cada fichero generado** (`<id_estacion>.txt`).
    - Para no saturar el sistema de archivos ni el ancho de banda del servidor, el acceso concurrente a la lectura estará restringido por un objeto **`Semaphore` con un máximo de 3 permisos simultáneos** (`new Semaphore(3, true)`).
    - Cada hilo deberá solicitar permiso con **`acquire()`** antes de abrir y leer el fichero, y liberar el permiso obligatoriamente en un bloque **`finally`** mediante **`release()`**.
- **Sección crítica y acumulación:**
    - La actualización de las estadísticas globales (total de muestras válidas, acumulado de partículas PM2.5 y conteo del nivel de calidad según el baremo)

constituye una sección crítica compartida por todos los hilos lectores. Debe protegerse mediante exclusión mutua (`synchronized` en método o bloque).

- **Control de estaciones sin respuesta:**
  - Si alguna estación sensora no llegó a generar su fichero, el hilo correspondiente registrará la incidencia e incrementará el contador de "estaciones sin respuesta".

---

2. Aplicación Estación Sensora (Proceso Hijo)

- **Arranque y presentación:**
  - Al arrancar, espera un tiempo aleatorio entre **0 y 500 milisegundos**.
  - Construye su identificador único a partir de una marca temporal del sistema: `System.currentTimeMillis()`.
  - Informa a la aplicación padre de su identificador imprimiéndolo por su salida estándar (`System.out.println(id_estacion)`).
- **Simulación de medición:**
  - Simula un tiempo de calibración y análisis durmiendo el proceso entre **1 y 3 segundos** (`Thread.sleep`).
- **Probabilidad de fallo (Sin respuesta):**
  - Para simular fallos de cobertura o averías en los sensores, de media **1 de cada 5 estaciones (20% de probabilidad aleatoria)** fallará y finalizará sin llegar a escribir ningún fichero.
- **Generación del fichero:**
  - Si la estación no falla, genera aleatoriamente un valor de concentración de partículas PM2.5 comprendido entre **1 y 100 µg/m³**.
  - Guarda dicho valor numérico en un fichero de texto denominado `<id_estacion>.txt` en el directorio de trabajo del programa.

Tabla de Niveles de Calidad del Aire (según PM2.5)

Rango de PM2.5 (µg/m³)	Nivel de Calidad	Categoría
1 – 12	Nivel 1	Buena
13 – 35	Nivel 2	Moderada
36 – 55	Nivel 3	No saludable para grupos sensibles
56 – 100	Nivel 4	Perjudicial

EJEMPLO DE EJECUCIÓN

Consola de la Aplicación Central (Ejecución con 8 estaciones)

```
None
Indica el número de estaciones sensoras a desplegar: 8
** Desplegando 8 estaciones sensoras... **
 -> Estación conectada: EST-1731094820112
 -> Estación conectada: EST-1731094820245
 -> Estación conectada: EST-1731094820389
 -> Estación conectada: EST-1731094820510
 -> Estación conectada: EST-1731094820674
 -> Estación conectada: EST-1731094820811
 -> Estación conectada: EST-1731094820930
 -> Estación conectada: EST-1731094821055

** Todas las estaciones han finalizado sus tareas. **
** Iniciando procesamiento con Semáforo (Max 3 hilos simultáneos) y Timer... **

[TIMER - Monitorización] Hilos de lectura activos: 3 | Permisos libres: 0
[TIMER - Monitorización] Hilos de lectura activos: 2 | Permisos libres: 1
[TIMER - Monitorización] Hilos de lectura activos: 1 | Permisos libres: 2

** Procesamiento finalizado. Cancelando temporizador. **

============================================================
           INFORME DE CALIDAD DEL AIRE
============================================================
Estaciones registradas : 8
Estaciones con respuesta: 6
Estaciones sin respuesta: 2

Desglose de niveles:
  - Nivel 1 (Buena)                              : 2
  - Nivel 2 (Moderada)                           : 3
  - Nivel 3 (No saludable para grupos sensibles) : 1
  - Nivel 4 (Perjudicial)                        : 0

Concentración media de PM2.5: 24.83 µg/m³
============================================================
```

CONSIDERACIONES A LA HORA DE VALORAR EL EJERCICIO

- **Requisito imprescindible de compilación:** El código entregado debe compilar. **Cualquier solución que no compile tendrá una calificación máxima de 2 puntos sobre 10.**
- **Acceso restringido:** No está permitido el acceso a Internet ni el uso de IA durante la prueba.
- **Buenas prácticas:** Nombres descriptivos de variables y métodos, separación modular del código en clases independientes, comentarios explicativos útiles y ausencia de código redundante o muerto.
- **Rendimiento y concurrencia:** Correcta utilización del `Semaphore` para limitar los accesos y sincronización limpia (`synchronized`) en las secciones críticas.
- **Gestión de temporizadores:** Uso estricto de `Timer` y `TimerTask`, asegurando el `cancel()` final para que la JVM termine su ciclo de vida.
- **Portabilidad:** Rutas y comandos configurados para operar tanto en Linux como en Windows sin errores de ruta o separadores.

RÚBRICA DE CALIFICACIÓN (TOTAL: 10 PUNTOS)

1. Aplicación Central (Total: 6 puntos)

- **1.0 pto:** Recuperación y validación del número de estaciones (parámetro/teclado con fallback a 5).
- **1.0 pto:** Creación de procesos hijo con `ProcessBuilder`, captura y registro del identificador desde el flujo de entrada.
- **1.5 ptos:** Gestión de hilos lectores concurrentes (`Thread` / `Runnable`), espera controlada con `join()` y coordinación general.
- **1.0 pto:** Implementación del `Timer` y `TimerTask` periódico de supervisión y cancelación limpia con `cancel()`.
- **1.0 pto:** Control de concurrencia en la lectura de ficheros mediante `Semaphore` (control de permisos con `acquire()` y `release()` en `finally`).
- **0.5 ptos:** Protección de la sección crítica con `synchronized` y presentación del informe estadístico final.

2. Aplicación Estación Sensora / Proceso Hijo (Total: 2 puntos)

- **1.0 pto:** Generación del identificador temporal y comunicación correcta hacia el padre por salida estándar.
- **1.0 pto:** Simulación de latencia con `Thread.sleep`, aplicación de la probabilidad de fallo (20%) y guardado correcto del valor numérico en el fichero.

3. Concurrencia y Calidad de la Arquitectura (Total: 2 puntos)

- **1.0 pto:** Despliegue de los procesos hijo y lectura simultánea de ficheros garantizando que no se producen condiciones de carrera ni cuellos de botella.
- **1.0 pto:** Estructuración del código en clases limpias, tratamiento adecuado de excepciones comprobadas (`IOException`, `InterruptedException`), portabilidad multiplataforma y documentación del código.